

Robot Localisation using Particle Filters

Third Year Individual Project - Final Report

April 2025

Theo Keetile

11028903

Supervisor

Joaquin Carrasco Gomez

Contents

1 Introduction	4
Background and Motivation.....	5
Aims and Objectives	6
Literature Review	6
2 Methodology	9
Particle filter working principles	9
Initialisation.....	9
Propagation	10
Weighting.....	10
Resampling	11
Estimation	11
Iteration	11
Simulation Setup	12
Pseudocode operation	16
Block Diagram	19
3 Results & Discussion.....	19
Changes to pre-set parameters	20
Number of Particles	20
Range of the lidar sensor	23
Noise prediction of the lidar sensor	25
Error in motion of the robot.....	27
Resampling techniques and improvements	30
Multinomial	30
Systematic	30
Residual	31
Stratified	31
4 Conclusion and Future Work.....	34
Suggestions for further performance tests	35
Increased map complexity	36
Introduction of moving objects in the known environment	37

Placement of robot in new environments..... 37

6 References 39

8 Appendices..... 41

Word count: 8700

Abstract

Robot localisation is a key part of mobile robotics, allowing autonomous systems to navigate through known environments. While Kalman filters are commonly used they often struggle when dealing with nonlinear movements and non-Gaussian noise. Particle filters, based on the Sequential Monte Carlo method, offer a more reliable alternative for tracking a robot's position on a 2D map.

A MATLAB simulation was developed to test this approach featuring a ballbot navigating from a starting point (1,1) to a target (9,9) within a virtual environment filled with obstacles and using a lidar sensor for distance detection. The particle filter tracked the robot's location by maintaining and updating a collection of weighted particles, using sensor fusion and resampling techniques.

Testing showed how various factors influence localisation accuracy. Increasing the number of particles improved convergence speed and reduced error. A wider lidar range captured more environmental details boosting accuracy. Introducing noise also led to greater localisation errors. Simulated odometry errors mimicking real-world conditions also negatively affected performance. Among the resampling strategies explored, systematic resampling offered the best trade-off between accuracy and efficiency.

The results highlight the effectiveness of particle filters in managing uncertainty making them a strong choice for localisation applications. High quality sensors and accurate motion models are essential for optimal results. These findings reinforce the potential of particle filters as a dependable localisation technique in autonomous robotics.

1 Introduction

Particle filters are a tool used in computer science and statistics to estimate the state of a system over a period. They work by allocating a set of particles to represent possible states of the system at a specific time. Each particle is weighted based on how well it matches the

observed data, and the particles are then resampled and updated at each time step to provide an improved estimate of the system's state.

This document describes the implementation of the particle filter in robotic applications. A robot is to be placed in a known environment and start its motion from one point to the targeted location. Throughout the motion, the particle filter is used in the approximation of the robot's location on the map until it reaches its target. In the real world, there are factors such as traction, mechanical indifferences in robotic parameters and noise on the measurement devices i.e., lidar sensor for positional awareness, which affect the accuracy of localisation. All factors will be taken into consideration to plot the assumed location of the robot using the particle filter and compare it to the actual position of the robot at any point in time. The difference between the two positions will describe the performance of the particle filter in localisation.

Background and Motivation

The main output obtained from a particle filter is an estimate and it is important to understand that an estimate is a rough judgement that has a quantifiable value or worth. The better the estimate, the more reliable the method. The development of the particle filter was sparked by the limitations of estimation models that came before such as Bayesian filters and Kalman Filters. The limitation of the Kalman filters is that it fails at estimating systems that are non-linear in behaviour and systems that have non-Gaussian noise induced on it. That is where Particle filters come and solve the estimation problem. An important implementation of the particle filter is in robotic localisation which is thoroughly discussed in this document. Other notable examples are in; weather forecasting where the complex state of the weather can be predicted in real-time [1], object and target tracking in military applications where positions and velocities of moving objects such as missiles and drones need to be monitored [2] and in asset prices and prediction of stocks in Economics need to be analysed [3].

Hospitals have patients moving to different areas around its perimeter for different reasons. With the development of machine learning and autonomous moving robots, the technology could be beneficial to use in the hospital and medical space. Objects such as hospital beds and wheelchairs could be fitted with autonomous robots using particle filter algorithms to

locate them as they move around to their targeted destinations. The key task would be, from an already known map of the hospital, move the robot from one point to another updating its location to a server that monitors its motion. This will in turn reduce the pressure of nurses needing to accompany and transport low risk patients around. It could also give the opportunity for unsupervised change of environment for patients granted by doctors. This relief of pressures on nurses should also help redirect time and resources to attending higher priority measures happening around the hospital.

Aims and Objectives

The key task is to design a robot localisation algorithm using a particle filter in a known map.

The milestones to be addressed for the successful evaluation of the task are as follows:

- Development of a virtual environment and map which the robot and particles can move and navigate.
- Development of a robot representation that can be provided with a motion.
- Developing a mathematical model for the robot's motion that will be used for setting a target and projected path.
- Implementation of sensor fusion algorithms on the robot in a virtual environment.
- Evaluation of the performance of the state estimation technique/ particle filter.
- Optimisation of the localisation procedure.
- Full evaluation of results obtained from the simulation and observations.

Literature Review

Robot localisation is a key factor of mobile robotics, enabling a robot to estimate its position and orientation which is collectively referred to as its pose within a given environment. This is essential for processes such as autonomous navigation, mapping, and interacting with a constantly changing environment. Since robots may have uncertainties in their movement and sensor readings, people usually rely on probabilistic techniques to tackle these localisation challenges.

Early methods of robot localisation relied heavily on Kalman filters, which model the robot's state using a Gaussian distribution and assume linear dynamics and Gaussian noise. While this approach is effective in controlled settings, these assumptions often fail in real-world scenarios where nonlinearities and non-Gaussian noise are more present. To fix the issues the Extended Kalman Filter with the general working principles described in [4] was developed, which linearises nonlinear system models around the current estimate using a first-order Taylor expansion. Further improvements led to the Unscented Kalman Filter being introduced which uses sigma-point sampling to capture the mean and covariance of the state distribution more accurately without needing to explicitly linearise [5]. Even with the improvements, both Extended Kalman Filter and Unscented Kalman Filter struggle with multi-modal distributions which is a common occurrence when the robot's initial position is unknown. This then paved the way for particle filters which is part of Sequential Monte Carlo methods that represent the robot's pose distribution using a set of weighted particles. Unlike Kalman filters, particle filters do not assume linearity or Gaussianity, making them way more adaptable for complex robotic systems. The basics of particle filters were outlined in [6], showing how Sequential Monte Carlo methods could work for Bayesian filtering but did not take off in robotics until later on.

One of the earliest and most influential applications of particle filters to robot localisation was Monte Carlo Localisation, proposed in [7]. This method uses a particle set to represent possible robot poses, propagating these particles based on a motion model and updating their weights using sensor data. A resampling step then concentrates particles in regions of high probability, helping the algorithm zero in on the true pose. Monte Carlo Localisation excels in global localisation where the robot must determine its position without prior knowledge and in handling the kidnapped robot problem, where the robot is abruptly relocated [8]. This further solidified the role of particle filters in robotics providing a detailed exploration of their theoretical and practical implementations [9].

Challenges such as particle depletion where resampling reduces diversity and risks losing the true pose hypothesis affect the particle filter. To mitigate this adaptive resampling techniques were introduced, triggering resampling only when the effective sample size (ESS) falls below a threshold and thus preserving particle diversity. Another interesting

development is the Rao-Blackwellized particle filter, which factories the state space to reduce computational demands by marginalising certain variables which is helpful in grid mapping applications [10].

Particle filters have also been adapted to deal with the Simultaneous Localisation and Mapping (SLAM) problem where a robot must simultaneously localise itself and build a map of an unknown environment. [11] introduced FastSLAM which is an algorithm that uses particle filters to estimate the robot's pose while maintaining separate Kalman filters for mapping features, therefore offering a computationally efficient solution to SLAM. An improved version FastSLAM 2.0 then addressed convergence issues by refining the proposal distribution [12].

Particle filters have been widely adopted across diverse robotic applications due to their flexibility. An example in [8] demonstrated robust Monte Carlo Localisation using laser range finders for indoor localisation which is a setup relevant to simulations involving structured 2D maps with obstacles. In underwater robotics where GPS is unavailable, [13] applied particle filters within a sparse delayed-state framework for view-based SLAM showcasing their utility in challenging environments. This is also seen in self-adaptive Monte Carlo Localisation using range sensors, improving localisation accuracy in dynamic settings [14]. These cases highlight the adaptability of particle filters to various sensor modes and environmental conditions.

Their computational complexity scales with the number of particles as this can become prohibitive in high-dimensional state spaces or in real-time applications. Techniques like Rao-Blackwellization help but the trade-off between accuracy and efficiency remains a concern [9]. Particle filters are sensitive to the accuracy of motion and sensor models where poorly tuned models can lead to divergence or failure. The kidnapped robot problem, while manageable with sufficient particles highlights the need for more robust recovery mechanisms as explored in the context of sensor resetting [15].

Researchers have been working to enhance particle filters by integrating modern techniques into their operation. The author of [16] proposed a deep learning-based particle filter leveraging neural networks to optimise the proposal distribution therefore improving

efficiency and accuracy. The fusion of traditional probabilistic methods with machine learning reflects a trend toward hybrid approaches in robotics. Additionally, distributed particle filters for multi-robot systems as investigated in [17], enable collaborative localisation opening new avenues for swarm robotics. Looking forward, advances in computational hardware and algorithms might help tackle scalability issues and potentially make particle filters suitable for 3D localisation or large-scale environments.

2 Methodology

The particle filter is going to be used to analyse and localise a robot moving in a known grid map. The robot is to be placed in an arbitrarily chosen starting position, given a motion as a set of instructions and it must follow towards the final position. The job of the particle filter is to estimate the position of the robot throughout the motion. If a person was to move from point A to point B following a set of directions, Global Positioning System GPS uses the communication between their device and a satellite to estimate the real position of the device. The particle filter algorithm has knowledge of the environment including landmarks and boundaries so when it is given the same input as the real-world robot, how well can the algorithm estimate the position of the robot throughout motion. The robot will be susceptible to disturbances in the forms of noise and errors in motion to give examples, and the particle filter design should account for how these disturbances will affect localisation to produce the best estimate of the robot.

Using ideas from [18], the particle filter works in the following steps.

Particle filter working principles

Initialisation

After setting up the simulation environment and determining the path of motion for the robot, the initialisation process begins which involves setting a parameter for the number of particles that are going to be used to represent the possible states of the system. The particles are scattered equally around the studied area/map as shown in Figure 1 because there is no prior knowledge or memory of the location of the robot.

Initialisation of the particles is represented in the equation below,

$$X_t = x_t^1, x_t^2, \dots, x_t^M$$

where X_t is a particle set at time t and x_t^i is the state/position of the particle with an index number of i in the particle set M at time t .

Propagation

At every time step, each particle is propagated forward by a dynamic control input to follow the path to the final position including an estimate of the noise present in the real system also determined in the initialisation stage. Particles are propagated using a motion model that mimics the robot's displacement calculated as the difference between consecutive robot positions (new robot position – previous robot position). The likelihood of a particle's position that has been moved is shown as,

$$p(x_t | u_t, x_{t-1})$$

where u_t represents the control input propagation and x_{t-1} represents the state of the particle x at the previous time step $t-1$. The expression in the form $p(a | b)$ describes the probability of particle a being at a new position following the occurrence of action b .

Weighting

This process represents the probability of the robot being at a specific position in the map. Based on the measurements of the particle filter system, the weight will represent the importance of the particle. The higher the weight, the more likely the particle reflects the state of the system which means that the robot is most probably located within the high weighted area. The following equation represents weight likelihood of the particle,

$$w_t^i = p(z_t | x_t^i)$$

where w_t^i is the weight of particle i at time t and z_t is the measurements obtained by the lidar sensors which are used to compute the weight of the particle. Weights are updated based on the likelihood of simulated lidar measurements matching the robot's actual position readings.

Resampling

Resampling is performed to remove particles that have low weights, which translates to a low likelihood of the robot's possible position. This is done at every time step to filter out 'irrelevant' particles which over time put an unnecessary strain on computational power. It then keeps high weight particles which are more useful for evaluation of the system state. Low weighting values are discarded and will not be used in computation or analysis of the robot's position in the next cycle. There are various resampling techniques present and the one chosen in this application of the particle filter is systematic selection method. This method ensures that high-weight particles, representing likely robot positions, are replicated, while low-weight ones are discarded, mitigating particle depletion.

Estimation

Post resampling, the state estimate of the system is represented by the weighted average of the particles positions. In this analysis this would represent the mean location of the high weighting particles across the map. The mean location is the output of the particle filter and will be used to evaluate the performance and accuracy of localising the robot. The equation below represents the position,

$$\hat{e}_t = \sum_i^M (w_t^i * x_t^i)$$

where $\hat{e}$ is the estimated position of the system at time t , which is obtained by summing the product of the particle's positions and weights to measure their importance. This will highly represent the mean location of the high-weight particles on the map. Uncertainty in localisation is visualised with a threshold ring in the implementation, calculated as the mean standard deviation of particle coordinates reflecting the filter's confidence.

Iteration

All steps are repeated at every time step to produce a continuous estimate of the system location which should ideally get closer and closer to the true position of the robotic system. The filter iterates over 100 steps, updating the robot and particles toward waypoints ([1,1] to [9,9]). The position of the new particle is described as,

$$x_{t+1}^i = p(x_t^i | u_t | w_t^i)$$

where x_{t+1}^i is the position of the particle i at new time $t+1$. The expression given describes the new position of particle i as a function of the old particle x_t^i which underwent an input u_t and has a weight of w_t^i being assigned to it.

Simulation Setup

The simulation for the operation of a particle filter is modelled using a map, robot and position measurement device, which are created as elements of a virtual environment for the robot to use to travel through.

Map

The map shown on Figure 1 is set up on the simulation platform to be used to evaluate the motion of the robot and performance of the particle filter.

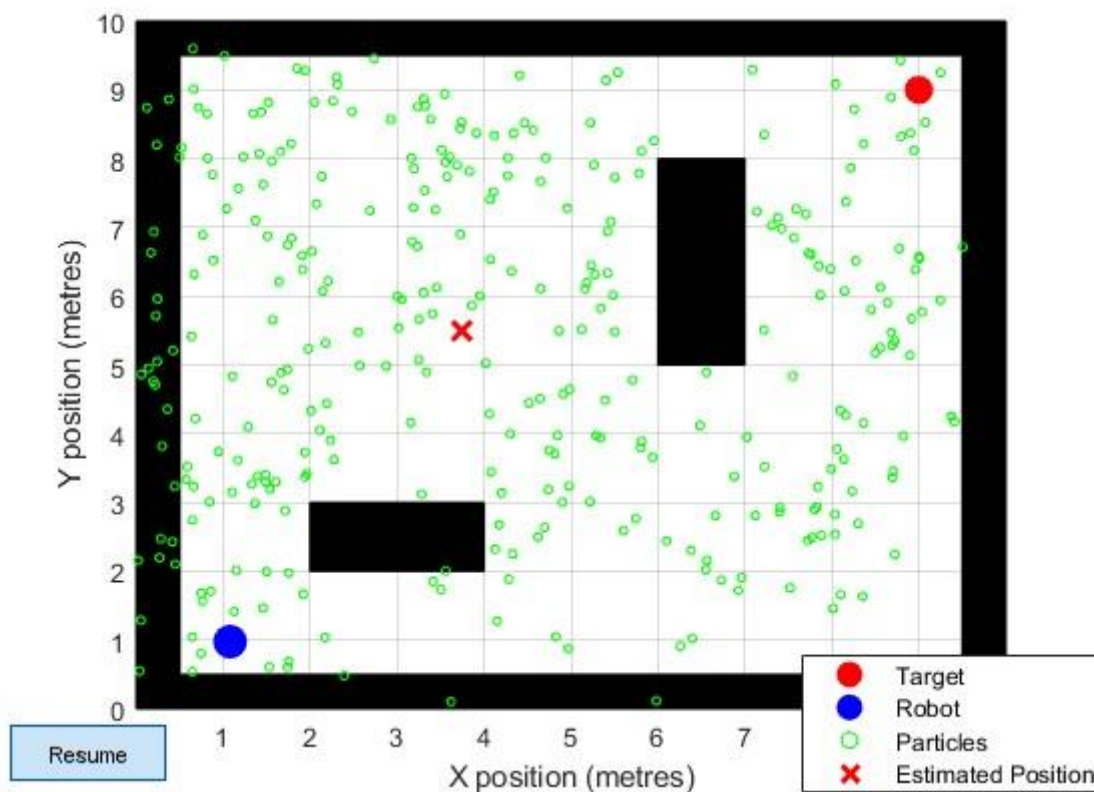


Figure 1: A map showing the simulated environment where the robot moves and particle filter performs localisation.

For the purposes of simple visualisation, a 2D map is chosen as shown in the Figure 1. The particle set (small hallow green circles) is continuously used to represent the distribution in the ‘belief’ of the state of the robot at every time step. This is represented in Figure 1 having the particles in the first iteration scattered randomly around the map. This illustrates the distribution of particles during one iteration if the loop before the robot (blue dot) reaches the final destination indicated by the red dot.

In the system designed, a choice of 500 particles is chosen to represent the possible states of the robot. The particle filter initialises 500 particles uniformly across a 10x10 grid, reflecting a global localisation scenario where the robot’s initial position is unknown. This choice balances computational efficiency with sufficient coverage of the state space. In this stage each particle is assigned an equal weight of $1/500$, forming a uniform distribution over the map representing no prior knowledge of where the robot may be. Values to simulate the anticipated error in the motion of the robot, noise in the lidar sensor measurement and range of the lidar sensor are established in the initialisation stage and can be varied to study the motion of the robot.

Robot

The robot in Figure 2 is simulated to be a solid circular object. The motion of the robot will be assumed to be that of a wheeled robot that has similar movement as described in [19] and Figure 2. This is a spherical wheel (omniwheel) allowing the robot to travel in any lateral direction without the need to change its initial orientation by turning. The central wheel has three or more ideal supporting wheels driving it within a feedback loop, and through this design the balance of the robot will be maintained and the velocity of travel can be varied throughout the motion.

The ballbot’s motion is more complex than that of a standard wheeled robot using a differential drive system of motion because it must continuously balance while moving. Its dynamics are similar to an inverted pendulum on a moving base, where the base is the spherical ball. For precise control, the full equation of motion is derived using Lagrangian mechanics, accounting for both the ball’s movement and the robot’s balancing described in [20]. From [20], “The angle between the body and the vertical is referred as the body angle

φ , while the angle between the ball and the body is referred as the ball angle ϑ " which results in a simplified representation of the equation of motion of the robot as,

$$M(q)\ddot{q} + C(q, \dot{q}) + G(q) + D(\dot{q}) = \begin{bmatrix} \tau \\ 0 \end{bmatrix}$$

where $q = [\vartheta, \varphi]$ is the generalised coordinate vector, $M(q)$ is the mass/inertia matrix, $C(q, \dot{q})$ is the vector of coriolis and centrifugal forces, $G(q)$ is the vector of gravitational forces, $D(\dot{q})$ is the frictional torque vector and τ is the torque applied between the ball and the body in the direction normal to the plane.

In comparison to a differential drive system with two wheels used to provide motion, the ballbot has advantages of manoeuvrability which allows it to navigate tight spaces without reorientation and a small footprint because of its single-point contact to the ground which helps it improve its agility. The disadvantages of choosing this type of robot odometry may be linked to its limitation on speed and terrain as the ballbot is best suited for smooth or flat surfaces while the differential drive performs well in different landscapes and can operate at higher speeds.

This will introduce errors in the odometry and motion of the robot, including errors in the reading of distance measurements which will be discussed in the Results & Discussion chapter of this document.

For the system at study, the robot and particle filter are both fed with the control input u to propagate every particle identically to the motion of the robot and with that, the robot and particles' position can be compared to process the particle weights. Gaussian noise with a standard deviation is added to simulate odometry uncertainties, such as wheel slippage or uneven surfaces. The updated particle position is given by,

$$x_t^i = x_{t-1}^i + u_t + N(\mu, \sigma)$$

where $N(\mu, \sigma)$ is the representation of the noise with mean value μ and standard deviation σ .

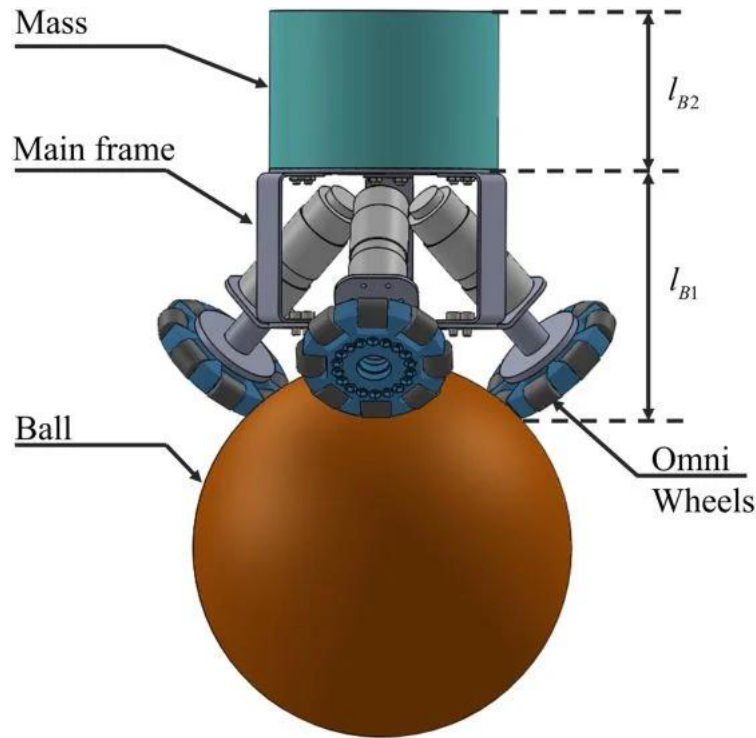


Figure 2: A ballbot [19] which is a robot with one wheel capable of balancing and moving in all directions in space.

Position measurement device

The distance measurements for the particle filter are taken through a lidar sensor. It uses infrared light rays which it emits and upon feedback from reflection with a surface, it gives the particle filter spatial awareness as the robot travels through the 2D map. A typical lidar sensor to be used in real life testing is like the one in datasheet [21]. It has a 270-degree aperture angle at 1-degree resolution and working range of 0.5 metres to 4 metres. The lidar rays bounce off the walls and obstacles present which represent targets and landmarks in the map shown in Figure 1. This process as illustrated in Figure 3 gives information which is processed as distance between the simulated robot and map parameters.

To get coverage of the area around the robot and the position of the robot in the simulation, 8 lidar sensors containing only 1 emitter and receiver circuit [22] are placed on the simulated robot circumference in the 8 cardinal point directions(N, NE, E, SE, S, SW, W, NW). The greater number the directions chosen, the more information the particle filter will have about the surroundings of the robot and provide a better estimation of the position. For each robot position, the 8 lidar rays are cast in the directions as part of a function to get

lidar distance measurements, with a maximum range of 2 units out of 10 the as maximum boundary length and noise standard deviation of 0.2 decided following tuning the value. The error between predicted position by particles and real position measurement by robot is computed as,

$$e = \sum (z_{\text{predicted}} - z_{\text{real}})^2$$

and weights of the particles are assigned using a Gaussian likelihood,

$$w_t^i = \frac{1}{\sigma\sqrt{2\pi}} * \exp\left(-\frac{e}{2\sigma^2}\right)$$

where $\sigma = 0.2$.

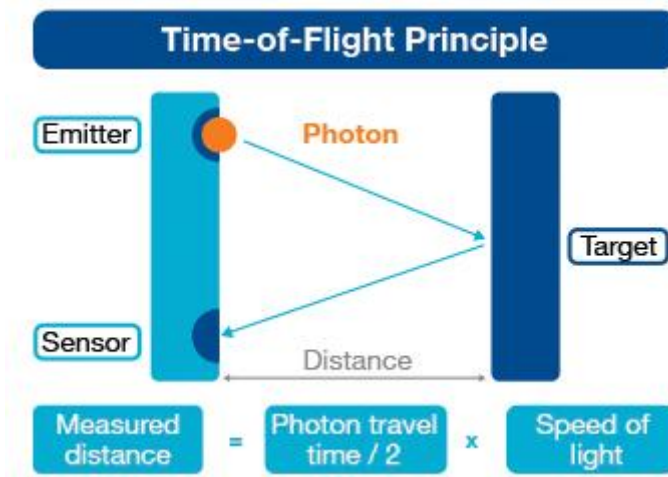


Figure 3: working principles of a lidar sensor using infrared light emission and a sensor to measure distance.

Pseudocode operation

The particle filter's operation split into the simple steps explained in the working principles section can be described by the following algorithms. From [23] the algorithms give a brief overview of how the particle filter works technically.


```

Require: A set of Particles for Robot  $i$  at time 0:  $S_i^0 = [\mathbf{x}_j, w_j : j = 1 \dots M]$ .
 $W = w_j : j = 1 \dots M$ 
while (Exploring) do
     $k = k + 1$ ;
    if (ESS( $W$ ) <  $\beta * M$ ) then {Particle Population Depleted (Equation 5)}
        Index = Resample( $W$ );
         $S_i^k = S_i^k(\text{Index})$ ;
    end if
    for ( $j = 1$  to  $M$ ) do {Prediction after action  $\alpha$ }
         $\mathbf{x}_j^{k+1} = \hat{f}(\mathbf{x}_j^k, \alpha)$ 
    end for
    s = Sense()
    for ( $j = 1$  to  $M$ ) do {Update the weights}
         $w_j^{k+1} = w_j^k * \mathcal{W}(s, \mathbf{x}_j^{k+1})$ 
    end for
    for ( $j = 1$  to  $M$ ) do {Normalize the weights}
         $w_j^{k+1} = \frac{w_j^{k+1}}{\sum_{j=1}^M w_j^{k+1}}$ 
    end for
end while
    {ESS is the Effective Sample Size, see Equation 5}

```

Algorithm 1: Particle Filter Algorithm; procedures are noted as underlined text, Comments are inside curly brackets "{comment}".

Algorithm 1 [8] describes a Particle Filter used for robot localisation which follows an iterative process to estimate the robot's position using a set of weighted particles. The key steps are:

1. Initialisation: A set of particles is initialised, each with a state (position) and weight.
2. Resampling: Resampling is performed to discard low-weight particles and replicate high-weight ones.
3. Prediction: Each particle updates its state based on the robot's motion model.
4. Sensing: The robot takes sensor measurements.
5. Weight Update: Particle weights are adjusted based on how well they match the new sensor data.
6. Normalisation: The weights are normalised to ensure they sum to 1.
7. Iteration: The process repeats until the robot completes its exploration.

This algorithm enables localisation by maintaining multiple hypotheses of the robot's position and refining them over time based on sensor feedback.

```

Input: Set of  $M$  Particles:: $S$ ; Translation distance:: $\rho$ 
 $\delta\rho = \frac{\rho}{K}$ ;
for ( $j = 1$  to  $M$ ) do { For each particle}
  for ( $k = 1$  to  $K$ ) do { At each of  $K$  steps}
     $E_{trs} = \text{rand\_N}(M_{trs} * \delta\rho, \sigma_{trs} * \delta\rho)$ ;
     $E_{drft} = \text{rand\_N}(M_{drft} * \delta\rho, \sigma_{drft} * \delta\rho)$ ;
     $\hat{\theta}[j] = \hat{\theta}[j] + E_{drft}$ ;
     $x[j] = x[j] + (\delta\rho + E_{trs} * \cos(\hat{\theta}[j]))$ ;
     $y[j] = y[j] + (\delta\rho + E_{trs} * \sin(\hat{\theta}[j]))$ ;
     $E_{drft} = \text{rand\_N}(M_{drft} * \delta\rho, \sigma_{drft} * \delta\rho)$ ;
     $\hat{\theta}[j] = \hat{\theta}[j] + E_{drft}$ ;
  end for
   $S'[j] = [x[j], y[j], \hat{\theta}[j]]^T$ ;
end for
Return( $S'$ )

```

Algorithm 2: Forward Translation with Noise; $\text{rand_N}(M, \sigma)$ is a pseudo-random number generator drawing samples from a Normal distribution with mean M and standard deviation σ ; procedures are noted as underlined text, Comments are inside curly brackets “{comment}”. The variables M_{trs} and M_{drft} represent the mean error and are experimentally derived.

Algorithm 2 [8] models the motion update step in a particle filter by simulating the forward translation of particles while incorporating noise. This helps reflect the uncertainty in a robot’s motion. The key steps in the algorithm are:

1. Input: The algorithm takes a set of M particles and a translation distance ρ .
2. Motion Update:
 - Each particle moves forward over K small steps, making the movement smoother.
 - Random noise is added to account for uncertainty in translation and direction.
 - The new x, y coordinates are computed based on the translation and heading angle θ .
3. Noise Injection:
 - E_{trs} - models translational noise from a normal distribution.
 - E_{drft} - represents drift in the heading angle.
 - The heading angle θ is updated at each step with drift noise.
4. Final Position Update:
 - After K steps, the final particle positions and orientations are recorded.

5. Output: The updated particle set S' is returned.

The difference in operation described in Algorithm 2 with the simulation we have are studying is the E_{drft} variable which describes the drift heading angle which is not a present independently but associated with the motion error including translational and drift error.

The programming platform of MATLAB R2024b has been chosen to simulate and test the hypothesis and will have graphs and figures generated and discussed in the document. The full MATLAB code is available in the appendix of this document for analysis of each subsection of the pseudocode.

Block Diagram

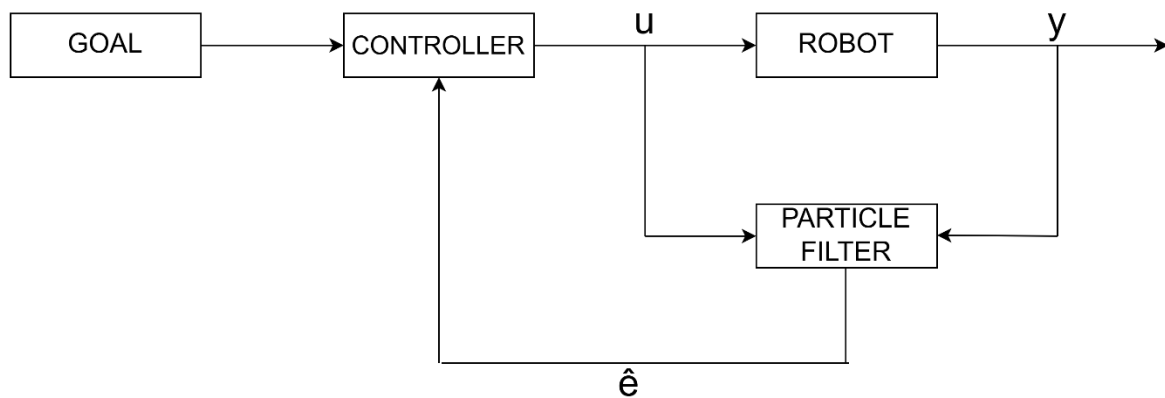


Figure 4: A block diagram that shows where the particle filter fits in localisation of the robot.

The block diagram in Figure 4 resembles a closed loop system. The GOAL is the target location that the robot must travel to through the provided trajectory. That target location is fed into the controller together with the estimated position $\hat{e}$ to compute a control input, U . The input is then fed into the robot and particle filter which produce the output y and estimated position $\hat{e}$ respectively.

3 Results & Discussion

The system at study is a simulation of real-world parameters which causes some uncertainty and assumptions when accurately representing the real world. The robot and the sensor are modelled only as simulations so not accurate measurements of actual dynamics are

obtained in this document. However, they can help to understand and mitigate real systems may face.

To test the performance of the particle filter, several parameters are modified from the ones predetermined such as the number of particles present in the map and the range of the particle filter just to give an example. The tests will be used to evaluate the optimal parameters to focus on when setting up a particle filter used for localisation, as well as the trade-offs the next designer of such a system must consider.

Key areas to be addressed in this section.

- Effects of the number of particles
- Limitations of the measurement device i.e. lidar sensor
- Evaluation of the effects of error in the robot motion
- Performance of the particle filter subject to noise in the lidar measurement and noise on the robot odometry
- Resampling techniques and their effectiveness

Changes to pre-set parameters

These parameters are set in the initialisation stage of setting up the particle filter. How they change will be studied alongside the performance of the particle filter. In this application, the particle filter is being used to accurately locate a robot moving in an environment so its accuracy in locating the robot and the number of steps it takes due to the changes of these parameters are what will be the deciding factors.

Number of Particles

The particle filter implementation comes with significant computational complexity because instead of studying the effect of 1 particle undergoing a particular motion and error dependant on the particle filter model, 500 particles have to be studied at the same time and decisions have to be made about their weight to proceed to the next steps.

To test the effect of the changing the number of particles used to represent the possible state of the particle filter, a test is conducted to check the speed at which the particle filter can locate the robot. It is expected that the larger the number of particles there are in space,

the faster the particle filter will converge to an average position on the map which represents the estimated position of the robot.

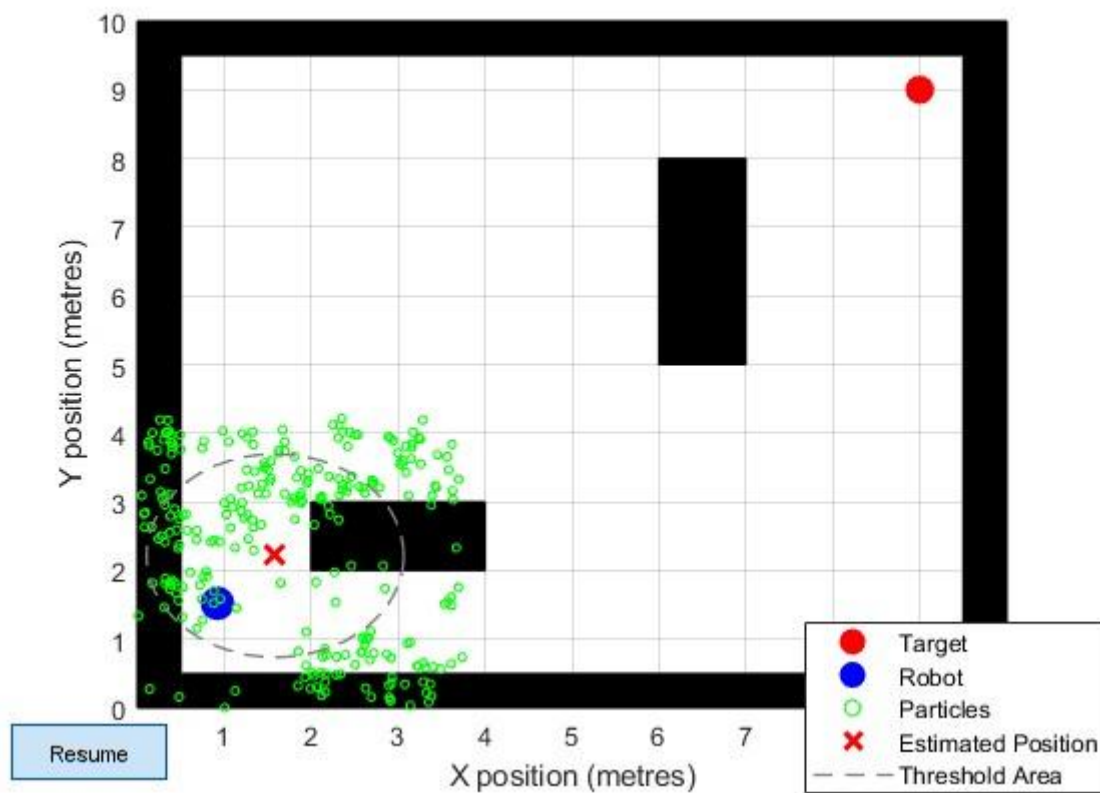


Figure 5: an image showing the area which the particle filter has confidence about the location of the robot.

Figure 5 shows the convergence of particles in the particle filter. There is a confidence level shown as the dotted ring around the red cross used to visually illustrate the mean location of the high weighing particles. Figure 6 shows the general trend of the error in localisation of the robot with time.

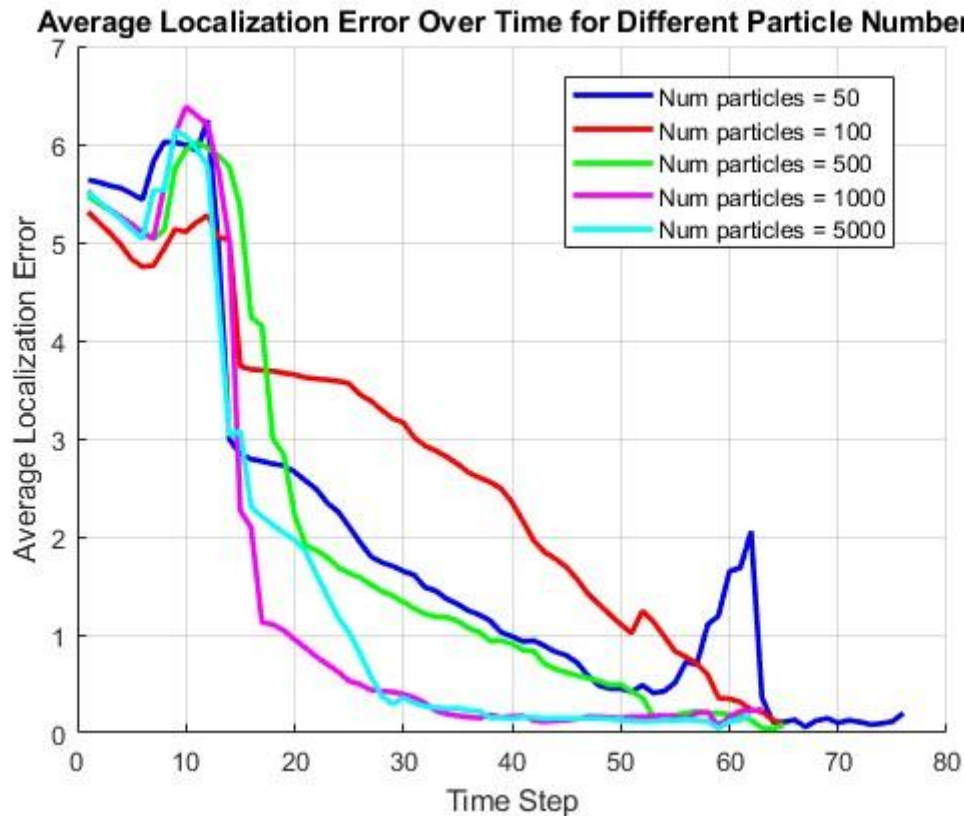


Figure 6: Localisation error over time for different number of particles.

The particle filter was set up with a fixed number of particles used to represent the possible location of the robot in the beginning of the localisation process. During the movement of the robot with time, the particle filter then uses the particles to locate the robot until it reaches the target location. This was done 5 times to get an average in trend of how well the particle filter works with different number of particles set in the initialisation stage. It can be observed from Figure 6 that as the number of steps increase the general error in localisation decreases which is expected because as the robot moves it is able to collect more data about the surrounding environment and have the particle filter's estimation converge to a mean location of where the robot may be in its motion.

Figure 6 suggests that the less the number of particles the more error present in locating the robot in the map, while the more the particles, the faster the particle filter 'finds' the robot on motion. The 50 particles and 100 particles trends show a slower decrease in efficiency of finding the robot while the 1000 and 5000 particle trends locate the robot within a much shorter time step range. The spike on between the 0 and 15th time steps for all trends indicate that the particle filter may still be trying to familiarise itself with the environment as

it gets lidar measurements from its surroundings and once it can identify landmarks. The rate of decrease in positioning error is directly proportional to the number of particles.

The following spike between 50th and 75th time steps on the 50 particles trend highlights the assumption that the less the number of particles the worse the particle filter performs, and it may indicate a moment where the filter loses confidence in its localisation due readings from a changing environment as the robot moves in the map.

Range of the lidar sensor

The lidar sensor is what gives the particle filter distance measurements of how far objects and obstacles are on the map. The range of the sensor represents how far the emitted rays can be projected and reflected to the sensor to give a reading. If the range is compromised, the particle filter will have a difficult time getting readings to help to decipher where the robot is to update particles and give an estimate of the robot's location.

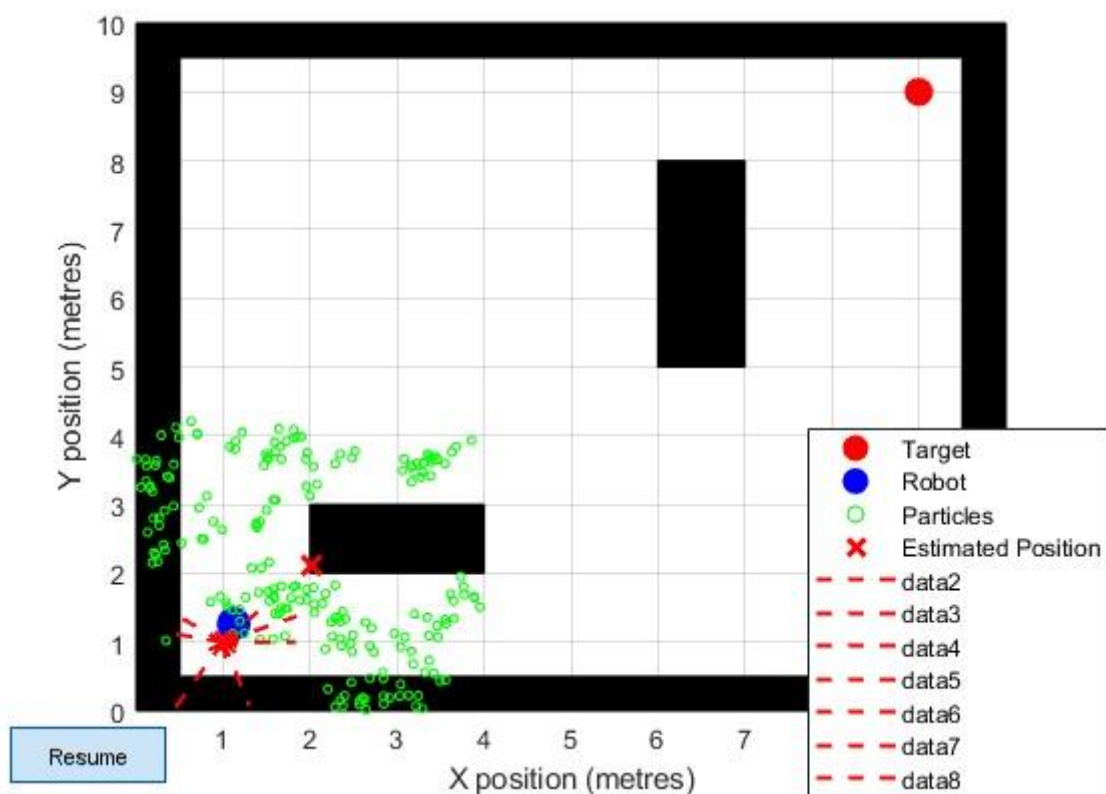


Figure 7: Beams of the lidar sensor shooting in all directions to estimate robot location.

Figure 7 gives a visual illustration of how the emitted rays from the lidar sensor may work and show which surfaces they reach dependent on the initial range set on the lidar sensor.

This simulation gives an idea of what sensor parameters a designer of such a system must be on the lookout for to match their implementation of the particle filter to their constraints. Figure 8 shows how the lidar range may affect the particle filter localisation result with the range increasing from low to high.

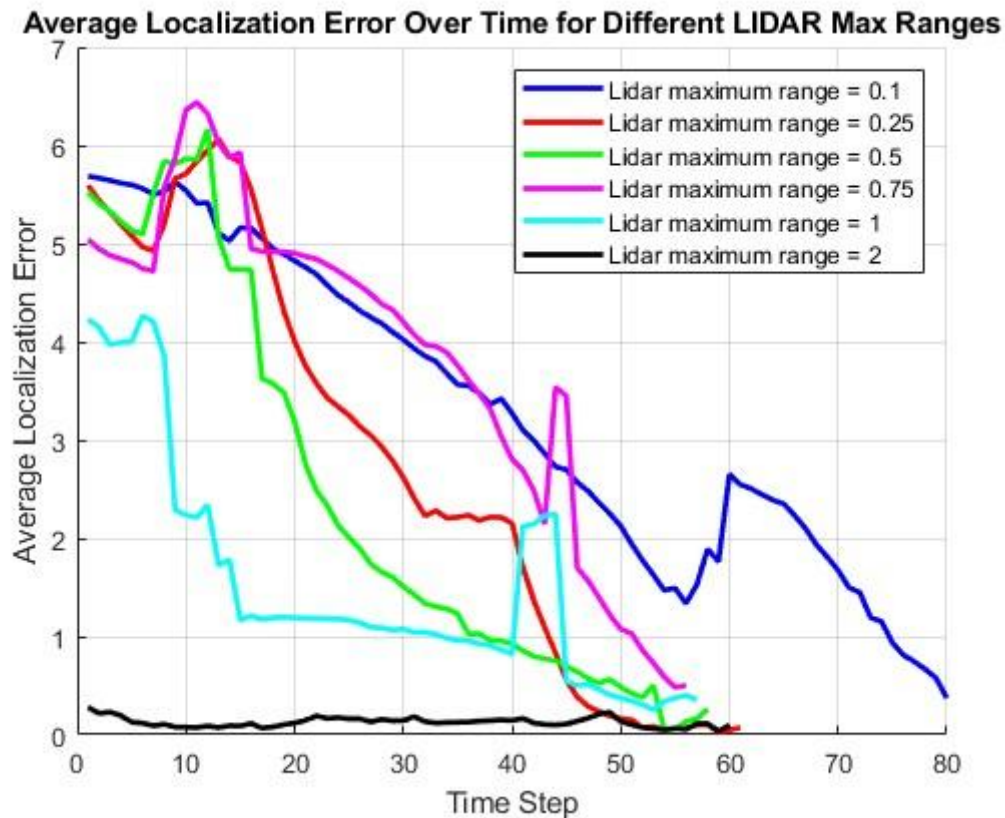


Figure 8: Localisation error over time for different lidar sensor ranges.

The shorter the range of the lidar sensor, the less surfaces that the sensor can identify. If surfaces along the planned trajectory of the robot are out of range of the lidar sensors, it would make it impossible for the particle filter to reference the location of the robot with any known landmarks or obstacles in the environment.

It can be seen in Figure 8 that when the maximum range of the sensor is the lowest at 0.1, the lidar sensor on the robot does not provide reliable data about the objects around as it goes through its trajectory which causes difficulty in localisation. This causes the number of steps needed to confidently localise the robot to increase compared to cases when the range is larger. The overall trends of the ranges from 0.1 to 1 unit are a negative gradient graph. They imply that as the robot moves through the map, the error in the localisation

decreases because the particle filter gets more readings from the lidar sensor which allow it to remove low weight particles and dial the position down to a certain area that the robot is in.

In the case of the lidar range being 2 units, Figure 8 shows that the particle filter knows exactly where the robot may be within the first few steps. This is ideal in the real world because there will be instant knowledge about where the robot is at all times but may be slightly affected when there is an obstruction introduced between the known environment landmarks and the robot. This is a simulation and in the real world, maps are much larger and there are limitations to how far sensors can operate making lidar range 0.5 a recommended value in our study to go off of when studying possibilities the real world may present.

Noise prediction of the lidar sensor

The use of sensors in the system introduces the possibility of error in the measurement of the distance between the sensor and the surface it interacts with. The measurement is taken as a mean value which has a deviation representing the possibility of error from the actual measurement. In this study, the noise present in the real world lidar sensor is modelled as a value which can be changed to reflect error deviations from the actual measurements.

Figure 9 is an averaged graph showing the effect of increase in the noise deviation with the number of steps it will take the particle filter to converge to 0.1 units of the actual position of the robot, showing confidence in localisation. The graph is linear with a positive gradient indicating that there is an increase in the average number of steps it takes for the particle filter to fall within the confidence zone of 0.1 units as the noise standard deviation increases. The error bars in the figure show the range in the number of steps it takes and grow with the increase in simulated error. The higher the error, the higher the possibility for a particle move into an area which is close to the robot location and hence shorten the localisation time explaining the final dip in gradient at lidar noise level 5.

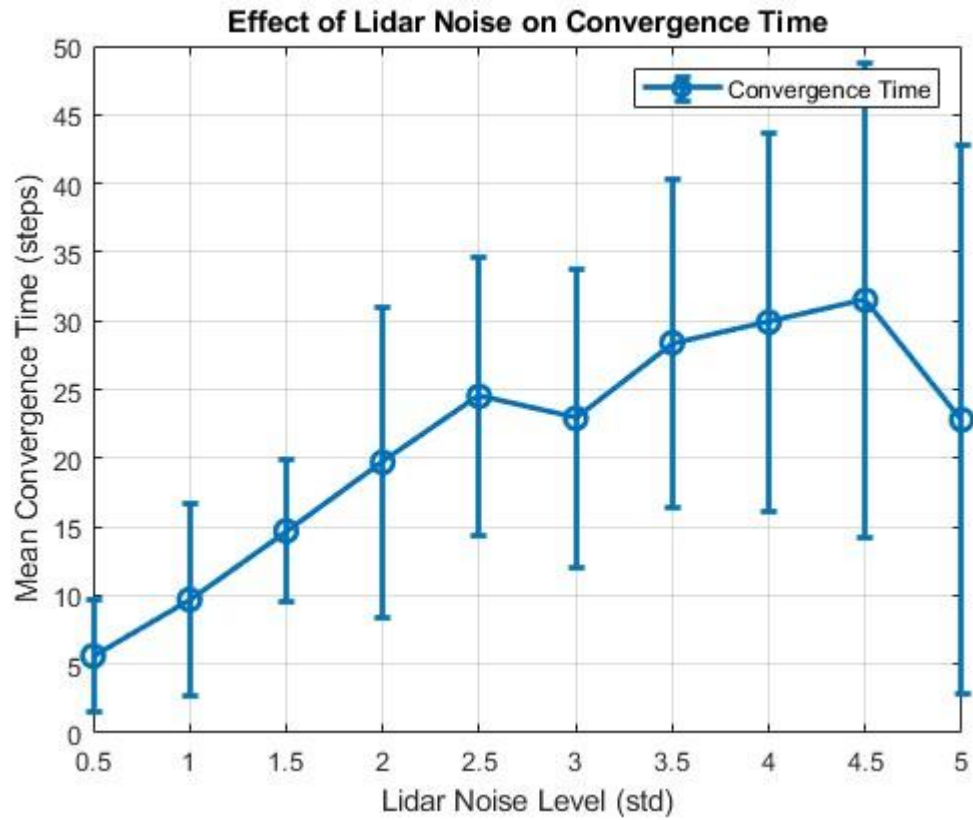


Figure 9: Effect of increase in lidar noise on locating robot in terms of the number of steps.

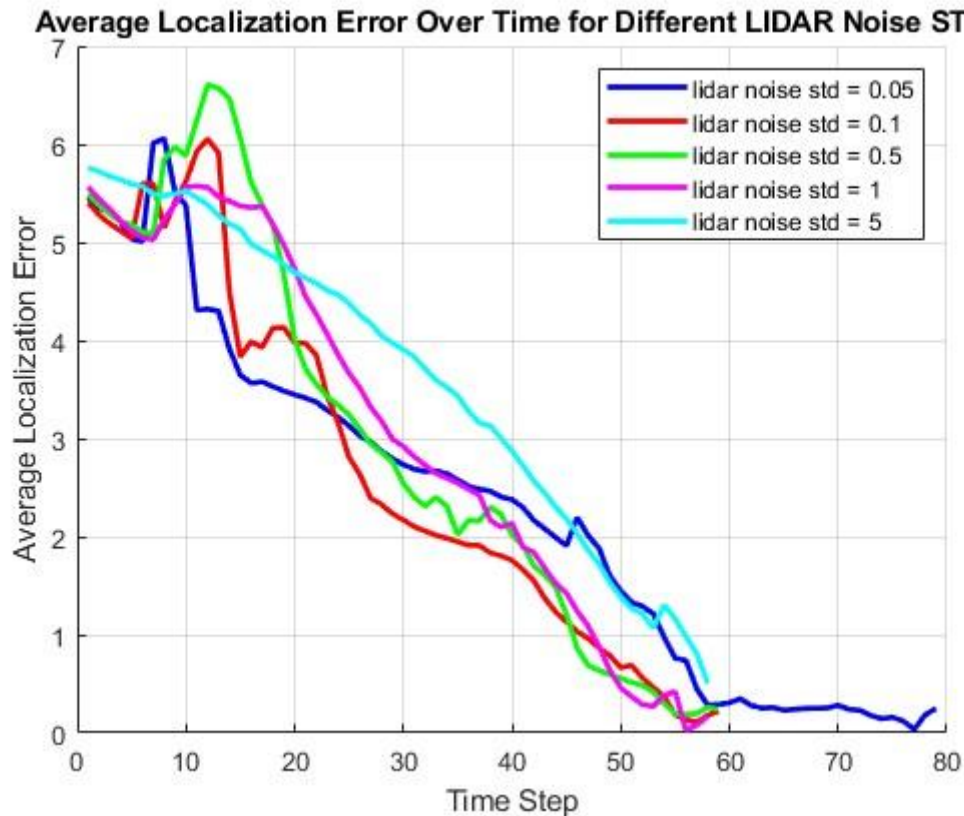


Figure 10: Trend showing the effect of increasing the error of the lidar sensor against the time it takes to localise the robot.

Figure 10 shows the trend the localisation process follows as the noise estimate of the robot increases. It can be noted from the graph that the lowest noise standard deviation value causes the longest number of steps needed to localise the robot. This is due to the suggestion that the particles are not likely to be within the small deviation area of the robot measurement. As the standard deviation increases there is a higher chance that the particle filter will localise the robot faster, but this trend does not increase the speed at which this is done, by much as shown in the similar shape of the standard deviations at values 0.1 to 1.

Error in motion of the robot

The ballbot [19] is unique in its dynamic movement. During propagation in the simulation from one point to another there is an error added to this motion. The errors added are due to the translational motion and errors due to drift resulting in deviation from the desired direction of movement as described in [21] which has the illustration in Figure 11. Based on

real world parameters such as friction and slip between the robot and the ground, the robot simulation accounts for the possibility of an error affecting the system.

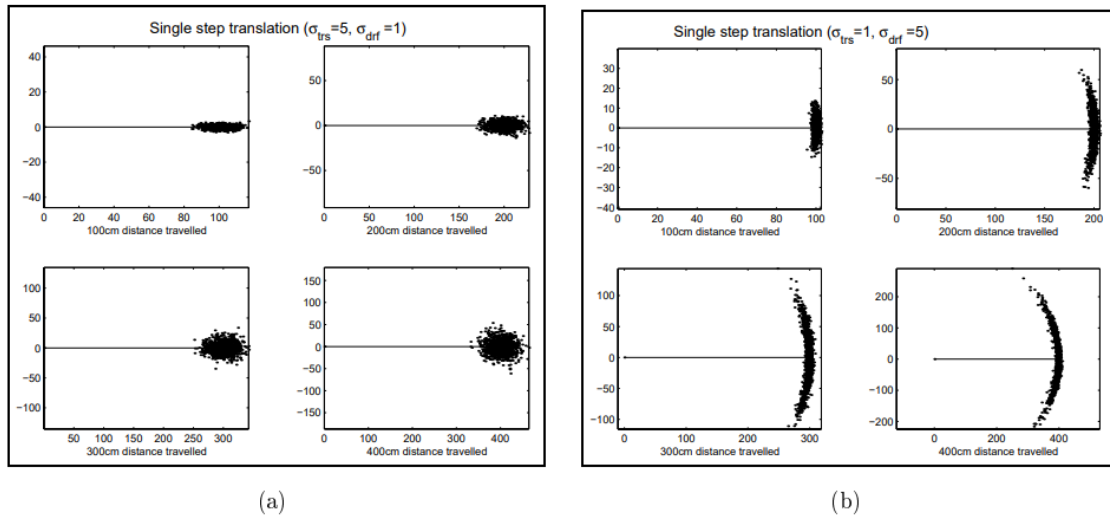


Figure 11: The effect of translational error (a) and error due to drift (b)

Figure 12 contains figures illustrating the different values of noise added to the simulated robot. The robot (blue dot) moves from coordinate position (1,1) to the final target location (9,9) represented by the red dot, through the path traced by the blue line. It can be observed that in figure (a) the noise added to motion was the least and gradually increased in figure (b). It makes it difficult for the particle filter to provide a reading for a location estimate if the robot has a simulated error much worse than figure (b). There is little to no accuracy in the estimated position because of the largely unpredictable error in the robot's movement.

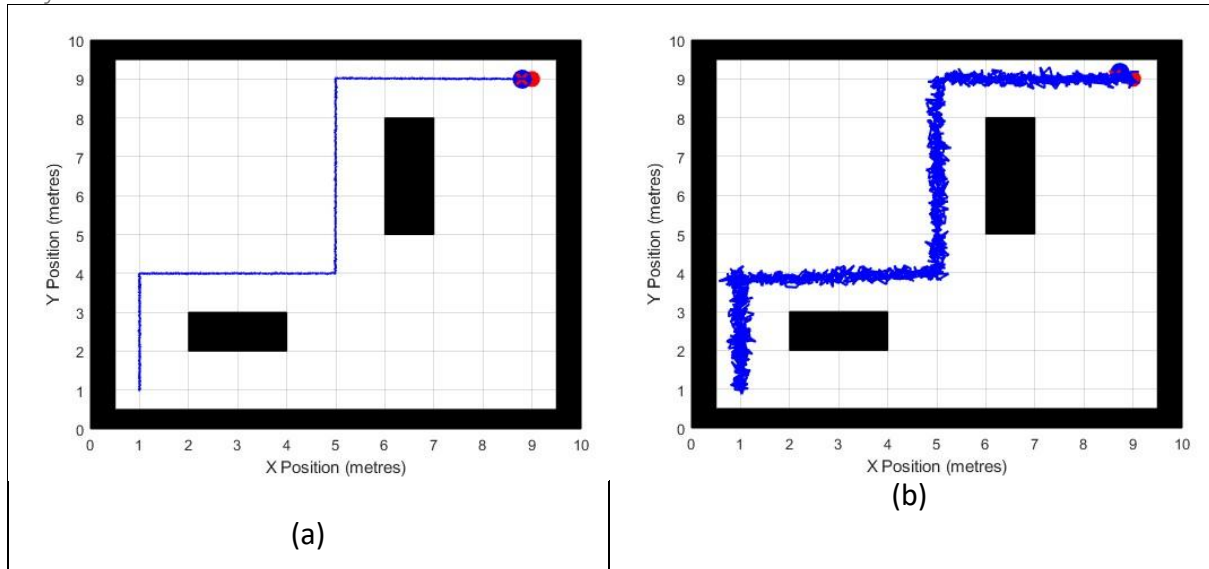


Figure 12: Visualisation of: (a)The motion standard error at an average of 0.01, (b)The motion standard error at an average of 0.1.

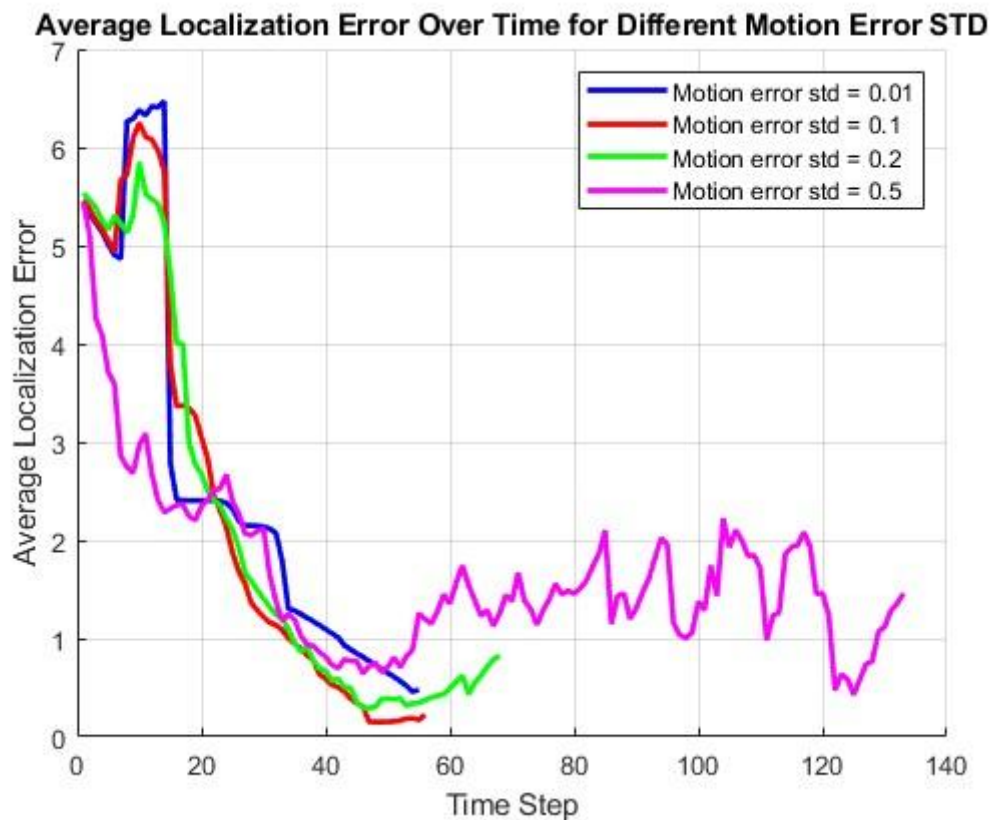


Figure 13: The average localisation error trend due to a change in simulated motion noise.

It is possible that there may be a permanent manufacture fault in the driving motors of the ball-bot which will cause a constant motion error. Figure 13 shows the trend the particle

filter follows when the motion error standard deviation variable is increased to simulate factors such as slippage and wear and tear on the ball-bot drive system. The combination of the permanent and conditional errors is anticipated in the design of the particle filter by studying the effect of changing the motion error as seen before in Figure 12.

Figure 13 shows a general decrease in the localisation error as the number of steps increases to show that the error inclusion may help put particles within the threshold area for estimating the position of the robot. The higher error deviations in motion cause an almost unstable error in localisation causing the particle filter to take a higher number of steps to locate the robot and not being ideal for a simulation setup. This translates to a possibility that if the error in motion in real life is above a certain value, it will cause the algorithm to fail in localisation and will need physical properties of the robot to be addressed and fixed to make use of the particle filter in localising the robot.

Resampling techniques and improvements

There are 4 main resampling techniques used in different applications in the operation of the particle filter. These are explained thoroughly in [24] and this subsection will briefly introduce the different methods highlighting their strengths and limitations.

Multinomial

This method randomly selects samples based on their weights. For analogy, imagine a carnival spinning wheel game where the weight of the particle determines the size of the section on the wheel. What essentially happens is that the more the wheel is spun at random, the more likely a particle will be chosen. During resampling this translates to particles with small weights are least likely to be chosen. This process being random causes a problem of the same particle being picked many times during resampling and some particles being overlooked completely and hence possibility of deviation.

Systematic

This method randomly starts at one point among the particle set and selects particles at regular step intervals. Unlike Multinomial resampling, instead of spinning the wheel independently for each pick as in the analogy, one random start is used and followed systematically. The downside to this method is that high weight particles might get picked

multiple times if the steps land in their range multiple times which causes domination and loss in variety of resampled particles.

Residual

This method ensures that high-weight particles get a fair share of being resampled first by duplicating them based on their weights. For analogy purposes, think about baking cookies and handing out whole trays to each particle according to its weight. Bigger weights will get more trays but only full ones and then from any leftover trays, particles are picked based on the fractional bits of their weights. This guarantees that big particles are not ignored. The downside of this method is that when particles have similar weights, it is difficult to identify which ones instantly go on to be resampled for the next step which can potentially eliminate the correct set of particles in the early stages.

Stratified

This method spreads out particle selections evenly by splitting the weight range into equal chunks and picking from each one. For analogy, imagine slicing a pie into equal pieces, one slice representing each particle and then randomly choosing a spot in each slice. This way, every part of the weight range gets a fair shot of being chosen keeping the resampled set diverse. This avoids big gaps or repeats but however the downside may be that it doesn't always select very high-weight particles as it should which may make it take longer to localise the robot.

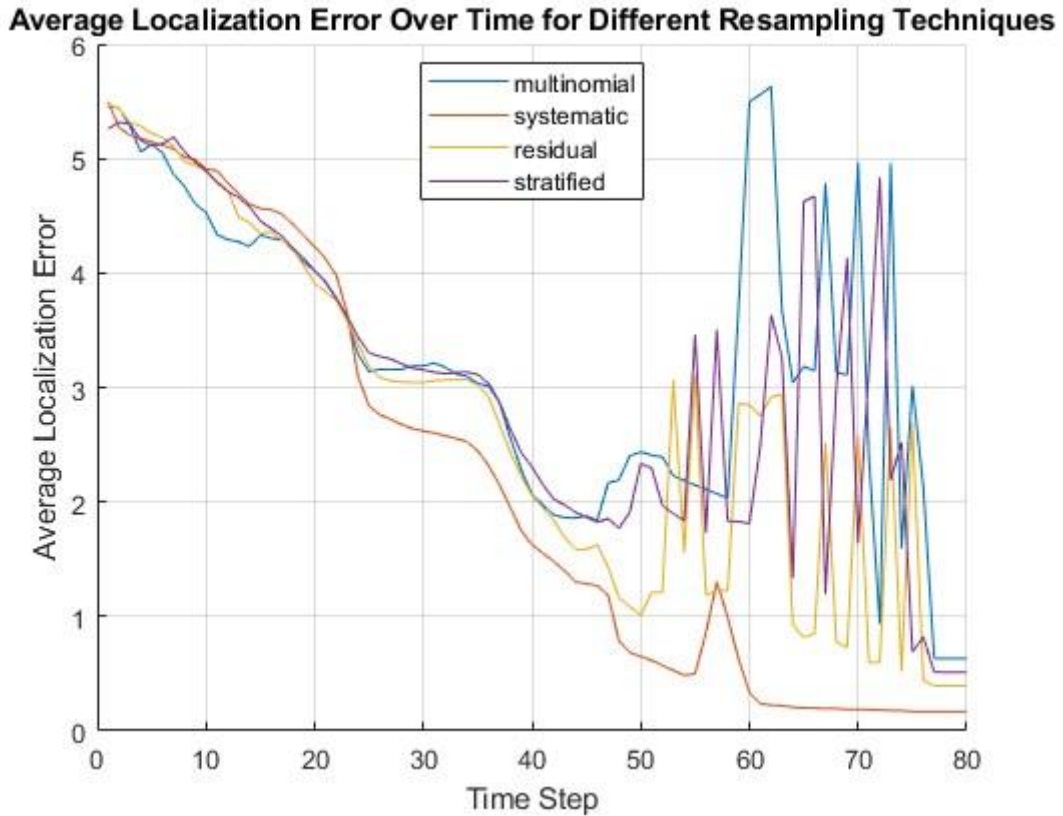


Figure 14: Accuracy of the resampling techniques applied to the same motion of the robot.

Figure 14 shows the performance of different resampling techniques used in localisation of the robot in this particle filter setup. The performance tests were repeated 5 times for each technique and the average localisation error vs time step were plotted. This plot indicates the amount of time in terms of steps each technique averages and by analysis systematic and residual sampling stand out to be the best resampling techniques while multinomial being the worst performing in this context. Systematic method seems to settle into a low localisation error earliest which is around 60 steps and maintains that until the end of the simulation. In comparison to the other techniques residual seems to do slightly better while multinomial and stratified evidently are the most unstable in terms of localisation based on the spikes they continuously get after 50 steps indicating difficulty in locating the moving robot.

Throughout the analysis in this report, systematic resampling was chosen because it ensures that particles are selected at evenly spaced intervals, which helps in maintaining a consistent spread of particles across the state space which is the simulation grid. This technique reduces the likelihood of degeneracy by avoiding the clustering of particles

around certain regions, which can occur in other resampling methods. Systematic resampling offers a more computationally efficient approach as it eliminates the need for random sampling within each partition which makes it suitable for scenarios where computational resources are limited. By providing a more uniform distribution of particles, systematic resampling improves the overall performance of the particle filter, ensuring better tracking accuracy and faster convergence, especially in dynamic environments with noisy sensor data.

At different times in the motion of the robot other resampling techniques may be more powerful than others. This leads to the reason to explore which resampling techniques may be best to use in the initial stages of localisation and which to use in the later stages of motion of the robot. Creating hybrid techniques which adapt dynamically to the motion of the robot may help in localisation of the robot. For instance, systematic resampling might lose diversity in a skewed distribution but pairing it with stratified or residual resampling earlier on maintains a broader particle set and leads to better long-term accuracy. This compilation of strategies may bring about two results i.e., the particle filter has improved the time it takes to localise the robot in its motion or the change of resampling methods may be detrimental to the localisation because of the increased computational complexity it has to deal with.

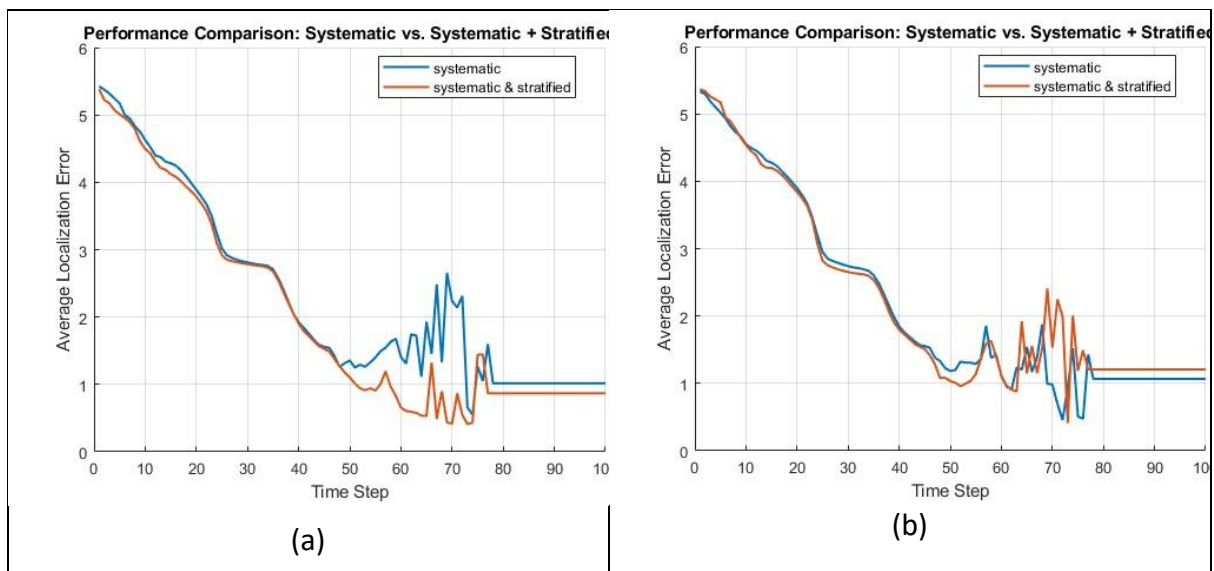


Figure 15: The performance test between systematic and a hybrid combination of systematic and stratified resampling in 2 different instances.

Figure 15 shows the performance test evaluated by averaging the localisation error which is the Euclidean distance between the true and estimated robot positions over 50 runs for each method. There is a variable which operates the switch between the systematic to the stratified sampling in the hybrid method. As the system tends to reach the final location around the 80th step, this variable is chosen to be 60 so that the strength of the systematic resampling which is its ability to maintain a uniform distribution of particles for longer countering degeneracy and stratified resampling's strength of allowing low weight particles to survive maintaining diversity in the later stages. This combination is meant to counter false localisation by allowing representation of both high and low particles continuously in the particle set.

Figure 15 therefore shows that there is little to no difference in using one resampling technique to using a hybrid combination of the resampling techniques as shown in both images (a) and (b). Image (a) favours the use of the hybrid resampling method as the hybrid graph levels out 2 steps faster and 0.2 noise levels lower than the single resampling method while image (b) favours the single resampling method in a similar manner. This is not a conclusive discovery, and it could therefore mean that the hybrid combination is not necessarily an effective solution when it comes to resampling of the particle filter in robot localisation applications so can be avoided in real time applications. The biggest challenge particle filters face is their computational demand, so this needs to be carefully managed by the designer.

4 Conclusion and Future Work

When building the real-life robotic system, the main contributors to the success of the particle filter in estimation of the robot are the quality of the odometry of the robot and the lidar sensor used in getting information about the surroundings. The more information the lidar sensor provides, the better the estimate of the position you will get from the particle filter. This translates to a linear relationship between the quality of the lidar sensor chosen and the estimation you get from the particle filter.

Through the MATLAB-based simulation, a virtual environment was successfully developed, with a robot represented as a ballbot, with a lidar sensor used in assessing the particle filter's performance in estimating the robot's position.

The experiments in the report illustrate critical findings, increasing the number of particles enhances the speed and accuracy of localisation of the robot which therefore comes at the expense of greater computational demand. A larger lidar sensor range improves the performance of the particle filter by providing more data about the environment. Noise present in lidar measurements increases the steps required for convergence which therefore highlights the importance of picking a quality sensor. Motion errors also degrade the localisation accuracy which leads to the need for precise odometry in the robot's design and tuning. Amongst the resampling techniques systematic resampling proved most effective in maintaining particle diversity and ensuring robust localisation.

The results and analysis section confirms that the particle filter has then ability to manage the nonlinear and non-Gaussian uncertainties which are mostly prominent in robotic motion systems, which then solve the limitations of simple Kalman and Bayesian filters.

The aim of the project was to develop an algorithm which can be used as a tool for applications of autonomous navigation in known environments like in hospitals and factories. The use of particle filters is computationally complex and the sensitivity in parameter tuning presents a challenge that must be carefully managed in practical implementations to achieve good performance.

Testing the particle filter in more complex maps or with moving obstacles would enhance its real-world application and building the robot physically would validate the simulation-based assumptions. In conclusion, this study demonstrates that particle filters offer a robust and versatile solution for robot localisation which is wholistic and can deliver accurate position estimates. With careful parameter altering and refining, particle filters hold a substantial promise for advancing autonomous robotic systems.

Suggestions for further performance tests

The map identical to Figure 1 used throughout this study is of a simple structure and used to mainly understand how the particle filter works in the easiest way possible but does not

marginally represent the complexity of the real-world robot areas. Autonomous robots used in factories for tasks such as inventory moving operate in environments that are highly complex and may change throughout the day.

Increased map complexity

Stock in a factory is stored in unique locations around the factory floor, and these must be stored or retrieved accurately when needed. Very often these factories have shelves filled with the stock with many isles placed and look like a maize from the top view as shown in Figure 16, which has a mixture of unique areas in the map and similar places.

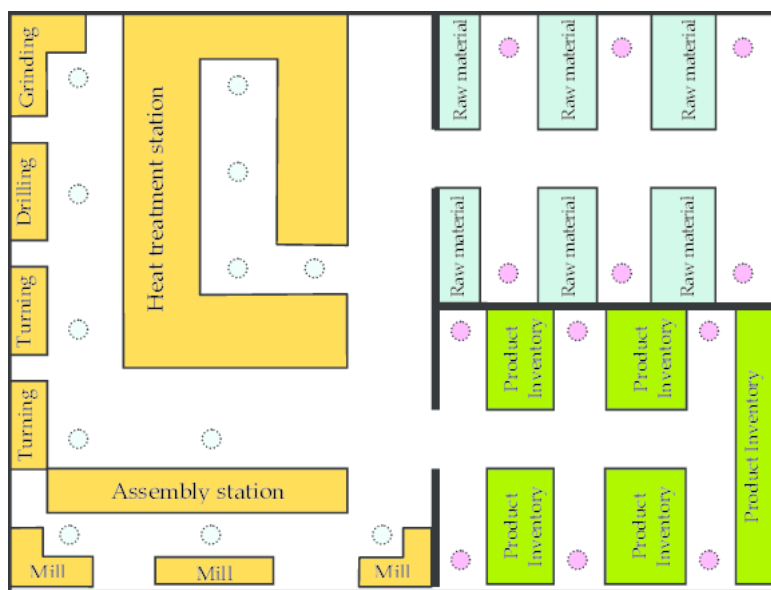


Figure 16: Example of the top view of a factory floor an autonomous robot moves in [25].

This will cause a problem with the localisation algorithm because if the environment the robot moves in is similar to another one, there has to be a way for the particle filter to locate which location it is specifically in. False localisation becomes prominent in such a situation and may lead to compromised decision making if the particle filter localisation output is depended on by another subsystem. When the robot is in continuous motion, there will be a reduction to the population of particles and cause particle deprivation as described in [18] which can cause localisation to fail. To counter such a scenario a threshold of particles to represent the robot can be maintained and replenished into the map when it goes below the value as described by Liu in [26] who represented the variable as the Effective sample size ESS.

Introduction of moving objects in the known environment

Human-robot interaction is bound to happen as the robot moves around the area that it is operating in. Large factories have fleets of autonomous robots roaming around the floors and they will come across each other and human workers during their operation. When a robot comes into close proximity with a moving object, at that instance the lidar sensor may be obstructed or make a distance measurement that does not match the layout of the map loaded in the particle filter. This is interpreted as a vague measurement of the surrounding and will not help in the localising process of the robot. Due to motion, the moving obstacle eventually moves passed into an area out of the range of the lidar sensor giving the particle filter an opportunity to get lidar measurements which are not interfered with. In such instances, the particle filter algorithm should be loaded with code that allows full re-scanning of the environment throughout the movement of the robot to reduce instances of failure to localise it due to the moving robots.

An improved particle filter approach from the one described in [27] which has sensors stationed around a closed space to provide location of the robot and to have all the robots on the floor space help each other in the localisation process through trajectory. This can be done by having their estimated locations uploaded to an on-site wireless central system and compared with each other as 'moving landmarks' when the robots interact, therefore improving the confidence of the localisation algorithm by providing distance information about each other compared to their surroundings.

Placement of robot in new environments

There will be instances when the robot is picked up and placed in a completely new location. This can be described as the 'kidnapped robot' problem. It requires the localisation algorithm to be still able to pick up where the robot is and locate it in the new area. Path and trajectory planning are not part of this study but if arrival to the same end goal is needed, the path will need to be recalculated as the particle filter locates the robot position. The most applicable approach to estimate the robot's position is to start the program from the beginning once again as a recovery mechanism where the full set of particles are used to locate the robot once again. If this is not possible, particle rejuvenation is advised where the

particle filter injects particles in random locations far from where the particle filter had converged to. This will allow the filter to not fail by converging in the wrong location but accommodate for such an instance where the robot has been completely moved to a new location and has to re-estimate where it is within a few steps of movement.

With the emergence of machine learning in the technological world, there will be a need to incorporate it into the localisation process. The system studied in this document is one that simulates reality, and this is only as good as the simulation and estimation made by the designer. Artificial Intelligence and machine learning can be added to the microcontrollers in the robots. With the robot being placed in unknown environments, variables such as the landscape including the friction between the robot and ground and how temperature affects components that build up the robot can be studied in real time to modify and adjust the particle filter estimation. This will use up more computational power and can cause a decrease in speed of performance of outdated processors but if the development of processors follows Moore's Law [28] which describes, chips reduce in size and improve in performance every year, it may allow this suggested computation to be done much faster in the near future.

The natural succession after analysing the operation of the particle filter on through analysis is to test the practical possibility of the proposed design system. This is possible to achieve by starting by setting an aim for integrating the simplest elements of the design together which are coming up with a design for the robot, identifying a sensor which will work the best in the implementation and coding the system. A significant amount of effort and time will be spent in testing and tuning the robot to get to move in the intended form. Only when that is achieved can reasonable results be logged and obtained for analysis and comparison to the proposed simulation in this document.

Robotics being a computerised simulation to imitate real life, a legged robot application can be studied to take the implementation of the particle filter to the next stage.

6 References

- [1] J. Poterjoy, "Progress toward the Application of a Localized Particle Filter for Numerical Weather Prediction," *Monthly Weather Review*, vol. 147, no. 4, pp. 1445–1469, Apr. 2019.
- [2] S. Arulampalam and N. Gordon, *Beyond the Kalman Filter: Particle Filters for Tracking Applications*. Boston, MA: Artech House, 2004.
- [3] P. Leung, C. S. Forbes, G. M. Martin, and B. McCabe, "Forecasting Observables with Particle Filters: Any Filter Will Do!," *Department of Econometrics and Business Statistics, Monash University*, Working Paper 22/19, Oct. 2019.
- [4] M. I. Ribeiro, "Kalman and extended Kalman filters: Concept, derivation and properties," *Inst. Syst. Robot.*, Tech. Rep. 43(46), pp. 3736-3741, 2004.
- [5] E. Wan, R. Van Der Merwe, "The unscented Kalman filter for nonlinear estimation," In *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (AS-SPCC)*, pp. 153-158, 2000.
- [6] A. Doucet, S. Godsill, C. Andrieu, "On sequential Monte Carlo sampling methods for Bayesian filtering," *Statistics and Computing*, pp. 197-208, 2000.
- [7] D. Fox, W. Burgard, F. Dellaert, & S. Thrun, "Monte Carlo Localization: Efficient Position Estimation for Mobile Robots," *AAAI/IAAI*, pp. 343-349, 1999.
- [8] S. Thrun, D. Fox, W. Burgard, F. Dellaert, "Robust Monte Carlo localization for mobile robots," *Artificial Intelligence*, pp. 99-14, 2000.
- [9] S Thrun, "Particle filters in robotics," In *Proceedings of the 18th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 511-518, 2000.
- [10] G. Grisetti, C. Stachniss, W. Burgard, "Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters," *IEEE Trans. Robot.*, vol. 23, no. 1, pp. 34-46, 2007.

- [11] M. Montemerlo, S. Thrun, D. Koller, B. Wegbreit, "FastSLAM: A factored solution to the simultaneous localization and mapping problem," In *Proceedings of the AAAI National Conference on Artificial Intelligence*, pp. 593-598, 2002.

- [12] M. Montemerlo, S. Thrun, "FastSLAM 2.0: An improved particle filtering algorithm for simultaneous localization and mapping that provably converges," In *Proceedings of the 16th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 1151-1156, 2007.

- [13] R. Eustice, H. Singh, J.J. Leonard, "Exactly sparse delayed-state filters for view-based SLAM," *IEEE Transactions on Robotics*, pp. 1100-1114, 2006.

- [14] L. Zhang, R. Zapata, P. Lépinay, "Self-adaptive Monte Carlo localization for mobile robots using range sensors," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 154-161, 2013.

- [15] S. Lenser, M. Veloso, "Sensor resetting localization for poorly modelled mobile robots," *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1225-1232, 2003.

- [16] J. Ma, Y. Cheng, H. Wang, J. Liu, "Robot localization using a deep learning based particle filter," *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5342-5348, 2020.

- [17] A. Martinelli, R. Siegwart, "Observability analysis for mobile robot localization," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1471-1476, 2005.

- [18] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, pp. 77, 2005.

- [19] A. Pham, B.H Thai , P.V Dang , N.T Vo , "Analysis of the parametric configuration impact on BallBot control performance". *International Journal of Mechanical System Dynamics*, 2024.

- [20] U. Nagarajan, G. Kantor, and R. Hollis, "Planning and control of a ballbot," *IEEE International Conference on Robotics and Automation*, pp. 4016-4021, 2010.

[21] Sick Sensor Intelligence, "2D LIDAR SENSORS," *TIM320-1131000S02 Datasheet*, Apr. 2024

[22] L. A. Wasser, "The Basics of LiDAR - Light Detection and Ranging - Remote Sensing," Earth Lab, Oct. 2024. [Online]. Available: [<https://www.neonscience.org/resources/learning-hub/tutorials/lidar-basics>]

[23] I. M. Rekleitis, "Particle Filter Tutorial for Mobile Robot Localization," Ph.D. dissertation, Centre for Intelligent Machines, McGill Univ., Montreal, QC, Canada, 2003.

[24] T. Li, M. Bolic and P. M. Djuric, "Resampling Methods for Particle Filtering: Classification, implementation, and strategies," in *IEEE Signal Processing Magazine*, vol. 32, no. 3, pp. 70-86, May 2015, doi: 10.1109/MSP.2014.2330626.

[25] E. Masehian and A. Hassannejad, "Graph-based multi robot motion planning: Feasibility and structural properties," *INTECH Open Access Publisher*, 2011, doi: 10.5772/12859

[26] J. Liu, R. Chen, T. Logvinenko, "A theoretical framework for sequential importance sampling and resampling", *Springer-Verlag*, Jan. 2001.

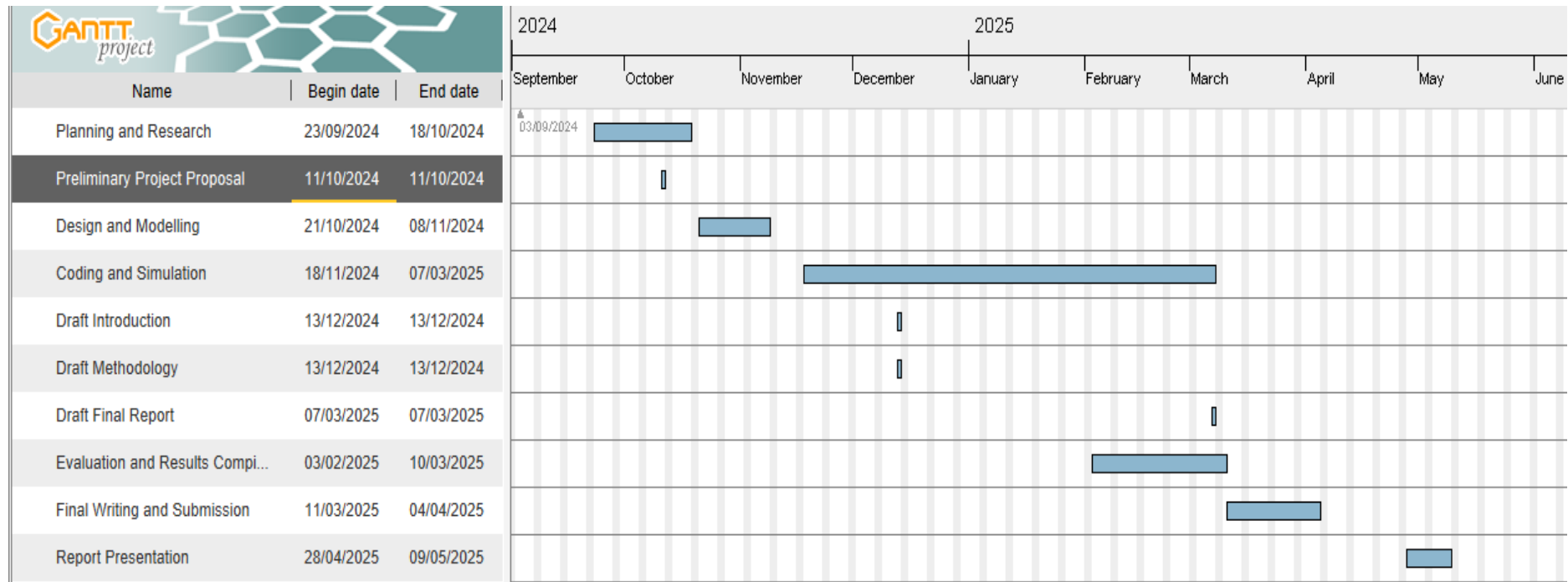
[27] P. A. Martínez, X. Lin, and M. Castelán, "A closed-loop approach for tracking a humanoid robot using particle filtering and depth data," *Intel. Serv. Robot.*, vol. 10, no. 4, pp. 297–312, 2017. doi: 10.1007/s11370-017-0230-0

[28] A. Kelleher, "Moore's Law – Now and in the Future," Intel White Paper, Feb. 2022. Available: <https://download.intel.com/newsroom/2022/manufacturing/Intel-Moores-Law-Investor-Meeting-Paper-final.pdf>

8 Appendices

A. Project Outline

Project Plan



Deliverables

- Preliminary Project Proposal & Risk Assessment - It includes a general outline of what the project is about, and the timeline tasks are going to be completed. The risk assessment lists all the risks associated with the type of project being undertaken. They both have a deadline of 1pm on the 11th of October 2024. No late submission penalty will be applied but it may affect the PRM1 mark as lack of engagement with the project.

- **Draft Introduction** - A 1000-word maximum introduction similar to the introduction in the final report has to be submitted to the supervisor. This will be analysed by the project supervisor and feedback will be provided in Semester 2 week 1. The deadline for the submission is 13th of December 2024 and no late submission penalty will be applied.
- **Draft Methodology** - The methodology section answers the how and why questions of the project. It answers to the development, design, implementation, background and motivation of the project. It will be analysed by the project supervisor and feedback will be provided in Semester 2 week 1. The deadline for the submission is 13th of December 2024 and no late submission penalty will be applied.
- **Draft Final Report** - This document must include all sections to be present in the final report. Content under all the topics may be incomplete but headings must be present. The supervisor will give written feedback on the table of contents section as well as 1 section in the report of the students choosing. The deadline for the submission is 7th of March 2024 and no late submission penalty will be applied but may affect the PRM4 mark.

Planning & Research (Week 1-4) - Defining the project scope, doing the background research, understanding the theoretical foundation and deciding on the technical approach.

Design & Modelling (Week 4-7) - Developing a dynamic model for the robot and implementing a model to combine the sensor into the algorithm

Coding and Simulation (Week 8-14) - Use MATLAB to write and simulate the robot in the intended behaviour to support our project outline.

Evaluation and Results Compilation (Week 14-17) - Use the obtained results from simulation to write up report and prove project undertaken was successful.

Final writing & Submission (Week 18-22) - Complete and submit the project having proved if you can fulfil your motivation or hypothesis.

Report poster Presentation (Week 23-24) - Present the project to supervisor and individual marker.

B. Risk Assessment

General Risk Assessment Form

Date: 11/04/2025	Assessed by: Theo KEETILE	Checked by: Joaquin Carrasco Gomez	Location: Manchester	Assessment ref no: 11028903	Review date: 11/04/2025
Task / premises: Research Project [Robot localisation using a particle filter] involving the design and simulation of a robot in a computer cluster.					

Activity (8)	Hazard (9)	Who might be harmed and how (10)	Existing measures to control risk (11)	Risk rating (12)	Result (13)
Use of screen(LCD) in writing and debugging code	Repeated / Prolonged or incorrect use	All users working with computer causing stress, eye strain, musculoskeletal issues, neck and back pain	Provision of an adjustable chair, adjustable screen height, suitable and sufficient lighting is maintained in each area Take regular breaks, ergonomic workspace setup, screen breaks, stretch exercises	Low	A

Activity (8)	Hazard (9)	Who might be harmed and how (10)	Existing measures to control risk (11)	Risk rating (12)	Result (13)
Computer Use	Computer overheating, electric shock, burns, fires, electrocution	All users working with computer workstations and electrically powered office equipment Personal injury – electric shock, electrocution and/or burns. Secondary injuries which may ensue	Ensure proper ventilation,	Low	A
Running simulations on a computer	Software crashes, loss of unsaved work, loss of code, data corruption	All users working with computer software can lose their progress unexpectedly.	Save work frequently, use and maintain backup systems	Low	A

Activity (8)	Hazard (9)	Who might be harmed and how (10)	Existing measures to control risk (11)	Risk rating (12)	Result (13)
Moving around in, and to and from the PC clusters / study areas	Trips, slips and falls any other injury which may occur in these locations	All users (normal working hours). Personal injury e.g. bruises, sprains and strains, slips, trips and falls (stairs, or tripping over objects), manual handling (e.g. PCs, desks, printer paper boxes)	Areas around computers must be kept free of trailing cables, floor areas are kept free from obstruction, adequate lighting is provided, and waste bins are supplied for rubbish. PC clusters, study areas and offices are inspected regularly. Stairwells and corridors are well lit.	Low	A

Activity (8)	Hazard (9)	Who might be harmed and how (10)	Existing measures to control risk (11)	Risk rating (12)	Result (13)
Lone working during normal working hours	Lack of or reduced access to first aid	Users of PC clusters, study areas or offices during low occupancy Personal injury and delayed medical attention	Avoid lone working and only use areas where there are other people within shouting distance. First aid support may be delayed if a person is lone working. Security can provide 1st aid and their contact number is on the rear of all University ID swipe cards and on the signage posted on doors to each PC cluster. Phone number is 0161 306 9966 Office users are made aware of this during safety induction sessions. For users with medical conditions that could be exacerbated by the delay of access to first aid the supervisor must seek the opinion of the person and if necessary the University Occupational Health service. A separate personal risk assessment may be needed.	Low	A

Activity (8)	Hazard (9)	Who might be harmed and how (10)	Existing measures to control risk (11)	Risk rating (12)	Result (13)
Lone working during normal working hours	Intruders	All users of computer clusters or offices who are lone working Becoming the subject of violence or aggression – stress, panic and injury	Avoid lone working and only use areas where there are other people within shouting distance. Users should have phone access to Security staff and should save this number in their mobile phones. During incidents of unease or suspicious activities, users should immediately go to a safe location and report to Security staff on 0161 306 9966. Details should later be shared with the Education Support Office/Safety Office.		

Activity (8)	Hazard (9)	Who might be harmed and how (10)	Existing measures to control risk (11)	Risk rating (12)	Result (13)
Lone working during normal working hours	Lack of support in the event of a fire Fire, smoke inhalation, burns	Users of PC clusters and offices who are lone working. Confusion, becoming lost, delay in exiting in an emergency situation – smoke inhalation, burns, unconsciousness and death	The evacuation procedure for the buildings is part of induction; this includes fire action notices, evacuation notices, and routes out of building, types of exit mechanism, fire alarms (audible and visual) and assembly points. All users receive an induction at the start of their programme. The contact number for Security is on the rear of the swipe card, this information is highlighted in the building inductions and on signage placed on the door to the clusters. Phone number is 0161 306 9966 All occupants must leave the building as soon as they hear the fire alarm by their most direct fire exit to the assembly point, quickly and calmly and proceed to the nearest assembly point and telephone Security. During an evacuation, face coverings should be worn while transiting but distancing may be temporarily ignored, as the duration is very short. Once outside the building social distancing should be resumed.	Low	A

Activity (8)	Hazard (9)	Who might be harmed and how (10)	Existing measures to control risk (11)	Risk rating (12)	Result (13)
Working from home	Poor posture, repetitive movements, long periods looking at DSE (display screen equipment)	Staff, students, visitors Back strain (due to poor posture). Repetitive Strain Injury (RSI) to upper limbs. Eye strain.	1. Set up workstation to a comfortable position with good lighting and natural light where possible 2. Take regular breaks away from the screen 3. Regularly stretch your arms, back, neck, wrists and hands to avoid repetitive strain injuries. Refer to workstation exercises here 4. Set up a desktop working space where possible and try to avoid working on a laptop without a docking station	Low	A

C. Continuing Professional Development

Continuing Professional Development Log

Name: *Theo Keetile*

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
MATLAB programming	Learnt how to design a functional robot in a simulated MATLAB environment	01/11/2024	200
Coding using AI assistance	Developed my quality of AI assisted problem solving to help with completing certain tasks	20/02/2025	25
Learnt how to use Github	This helped with the documenting progress in my code development and improve productivity.	01/03/2024	5
Report Writing	Improved my proficiency in report writing for different purposes including using IEEE standards of referencing	01/02/2025	70
Technical problem solving	Gained confidence in picking and choosing components that could possibly be used in my robot	01/02/2025	5

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
Learning Python	Learning how to program in python and possibly R programming language to broaden my ability in programming	Increasing adaptability and employability	01/07/2024
Modelling and Simulation	Using software packages such as SolidWorks and Blender to design complex components and systems. This can also include animation	Modelling and simulation of full systems to aid testing	01/07/2024
Integration of robotics and automated systems	Studying a post graduate course on improving effectiveness and production using robotics and automation of systems	How to leverage and integrate automation of current systems	01/09/2025
Improving how to use AI	Learning how to leverage AI and improve my productivity in the evolving world of digitalisation	Effective use of AI to better my life and productivity	01/06/2025
Engineering Management	Take on a master's or post graduate training to learn about engineering decision making on a national and international scale to allow more fair and informed decisions to be made by deciding boards	Shortage of engineers in governmental positions to address and decide on issues affecting engineers	01/09/2025

- D. Particle filter implementation in robot localisation, MATLAB programming language
example: [**https://github.com/TheoKeetile/Robot-localisation-using-a-particle-filter-**](https://github.com/TheoKeetile/Robot-localisation-using-a-particle-filter-)